My experience with Guard and related automatic test tools is that they can be useful for focused unit tests but not so useful for integration tests. It is tricky to have specific files know which integration tests they might be a part of without rerunning the entire integration-test suite on every file change. If you are willing to put a little effort into tweaking the Guardfile, though, you will not have to do anything to get the tests you are working on to continue running.

Writing Faster Tests by Bypassing Rails

Another way to run tests without loading the entire Rails framework is to write tests that don't require the entire Rails framework. That sounds crazy, I know, since you're writing a Rails application. Bear with me.

The tests you've written so far mostly fall into five groups:

- End-to-end tests. These tests require the entire Rails stack since you're testing the entire stack.
- Tests that use only ActiveRecord and need the database. Right now this group includes tests for ActiveRecord finder methods and your action object tests that save to the database.
- Request tests that require ActionPack and may or may not require ActiveRecord and may or may not touch the database, depending on how they're stubbed.
- Tests that use ActiveRecord objects but don't touch the database.
- Tests that use no Rails-specific classes at all.

The first category will, broadly speaking, be slowest, and each step down should be a faster set of tests. Right now you don't have many tests in the last category—I think the only ones there are the AvatarAdapter tests you wrote to interact with Twitter. How do you move tests to the lower and faster categories in this list? And why is removing Rails helpful?

Prescription 47	Rails is not your application; it is a framework on which you build your application—unless you work at Basecamp and actually develop Rails.

I'm indebted to Corey Haines and Gary Bernhardt for the ideas in this section. I particularly recommend you check out Corey's book *Understanding the Four*